BÁO CÁO THỰC HÀNH

Thực hành Nhập môn Mạng máy tính

Lab 2: Mật mã học hiện đại

GVTH: Nguyễn Ngọc Trưởng

Ngày báo cáo: 24/03/2026

Nhóm 13 – IT005.Q22.1

THÔNG TIN CHUNG

MSSV	Họ và tên	Nội dung báo cáo	% công việc
24521388	Nguyễn Hoàng Phúc	Câu 1, câu 2	50%
23520536	Phạm Cao Minh Hoàng	Câu 3, câu 6	50%

1. Nhiệm vụ 2.1

- Mã nguồn:

```
def F(right, subkey):
    return (right ^ subkey) & 0x0F

def feistel_round(L_in, R_in, subkey):
    L_out = R_in
    R_out = L_in ^ F(R_in, subkey)
    return L_out, R_out

def track_avalanche(msg, key):
    L, R = (msg >> 4) & 0x0F, msg & 0x0F
    subkeys = [key & 0x0F, (key >> 4) & 0x0F, (key + 1) & 0x0F, (key + 2) &
0x0F]
    print(f"Khởi tạo: L={format(L, '04b')}, R={format(R, '04b')}")
    for i in range(4):
        L, R = feistel_round(L, R, subkeys[i])
        print(f"Vòng {i+1}: L={format(L, '04b')}, R={format(R, '04b')}")
    return (L << 4) | R

print("--- Mã hóa M1 ---")
track_avalanche(0xAB, 0x12)
print("\n--- Mã hóa M2 ---")
track_avalanche(0xAC, 0x12)
```

- Các hàm thành phần chính:
  + Hàm F(right, subkey): Là hàm giả ngẫu nhiên F trong mạng Feistel, thực hiện phép toán XOR giữa nửa R và khóa con, sau đó AND bit với 0x0F để giữ lại 4bit.
  + Hàm feistel_round: Mô phỏng vòng lặp Feistel theo đúng định lý toán học:

$$LE_i = RE_{i-1} \text{ và } RE_i = LE_{i-1} \oplus F(RE_{i-1}, K_i).$$

- Các bước thực hiện:
  + Khởi tạo khóa và 2 bản rõ M1 và M2.
  + Tách bản rõ 8-bit ban đầu thành 2 nửa: Left 4-bit và Right 4-bit thông qua phép dịch bit >> 4.
  + Khởi tạo vòng lặp chạy 4 lần. Tại mỗi lần lặp, gọi hàm feistel_round để hoán đổi chéo L và R, đồng thời in kết quả L, R ra màn hình để theo dõi sự biến đổi.
  + Ghép L và R lại ở vòng cuối để thu được bản mã hoàn chỉnh.
- Kết quả thực thi:

```
PS C:\Users\admin\Documents\HK2_Nam2_2025-2026\NT101\TH\Lab01>
hon.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '
Lab02_Nhom13_Resource\Lab02_2.1.py'
--- Mã hóa M1 ---
Khởi tạo: L=1010, R=1011
Vòng 1: L=1011, R=0011
Vòng 2: L=0011, R=1001
Vòng 3: L=1001, R=1001
Vòng 4: L=1001, R=0100

--- Mã hóa M2 ---
Khởi tạo: L=1010, R=1100
Vòng 1: L=1100, R=0100
Vòng 2: L=0100, R=1001
Vòng 3: L=1001, R=1110
Vòng 4: L=1110, R=0011
PS C:\Users\admin\Documents\HK2_Nam2_2025-2026\NT101\TH\Lab01>
```

Hình 2.1.1 Kết quả chương trình

Nhận xét dựa trên kết quả in từ terminal:

- Em thấy rằng 2 bản rõ M1 và M2 khác nhau 1 bit cuối cùng của byte
- Phân tích dựa trên quá trình chạy:
  +Khối R của 2 mã bị xáo trộn thành 0011 và 0100
  +Ở các vòng tiếp theo, sự thay đổi ảnh hưởng tới cả L và R thông qua phép toán XOR
- +Đến vòng cuối cùng: kết quả của M1 là L=1001, R=0100 và M2 là L=1110, R=0011 ➔ 2 chuỗi bit đầu ra lúc này đã khác biệt hoàn toàn.

Kết luận: Kết quả này không thể dự đoán, phù hợp với mật mã học hiện đại trong việc giúp chống lại các phương pháp phân tích mật mã sai biệt.

2. Nhiệm vụ 2.2

- Mã nguồn:

```
- from Crypto.Cipher import AES
- from Crypto.Util import Counter
-
- key = b'1234567890123456'
- plaintext = b"UIT_LAB_UIT_LAB_UIT_LAB_UIT_LAB_"
- iv = b'0000000000000000' #
-
- # 1. AES-ECB
- cipher_ecb = AES.new(key, AES.MODE_ECB)
- ct_ecb = cipher_ecb.encrypt(plaintext)
```

```
-
-  # 2. AES-CBC
-  cipher_cbc = AES.new(key, AES.MODE_CBC, iv)
-  ct_cbc = cipher_cbc.encrypt(plaintext)
-
-  # 3. (CFB, OFB, CTR)
-  cipher_cfb = AES.new(key, AES.MODE_CFB, iv=iv,
   segment_size=128)
-  ct_cfb = cipher_cfb.encrypt(plaintext)
-
-  cipher_ofb = AES.new(key, AES.MODE_OFB, iv=iv)
-  ct_ofb = cipher_ofb.encrypt(plaintext)
-
-  cipher_ctr = AES.new(key, AES.MODE_CTR,
   counter=Counter.new(128, initial_value=0))
-  ct_ctr = cipher_ctr.encrypt(plaintext)
-
-  # Hàm in kết quả
-  def print_blocks(name, ct):
-      print(f"[{name}]")
-      print(f"Block 1: {ct[:16].hex()}")
-      print(f"Block 2: {ct[16:].hex()}\n")
-
-  print_blocks("AES-ECB", ct_ecb)
-  print_blocks("AES-CBC", ct_cbc)
-  print_blocks("AES-CFB", ct_cfb)
-  print_blocks("AES-OFB", ct_ofb)
-  print_blocks("AES-CTR", ct_ctr)
```

Hình 2.2.1 Source Code

- Thành phần chính:
  + Thư viện pycryptodome
  + Các đối tượng Cipher được khởi tạo tương ứng với 5 chế độ: ECB, CBC, CFB, OFB và CTR.
  + Hàm print_blocks: Hàm hỗ trợ định dạng chuỗi Hexadecimal đầu ra, tự động cắt chuỗi 32-byte thành 2 khối - mỗi khối 16-byte để dễ đối chiếu.
- Các bước thực thi:

+ Khai báo khóa Key (16 bytes), Vector khởi tạo IV (16 bytes, gồm toàn số 0) và chuỗi bản rõ đầu vào dài 32 bytes (2 block UIT_LAB_ lặp lại).
+ Lần lượt nạp Key và IV (đối với các mode yêu cầu) vào module mã hóa AES để cấu hình thuật toán.
+ Gọi phương thức .encrypt() cho từng đối tượng Cipher để tiến hành mã hóa bản rõ thành bản mã (dạng bytes).
+ Sử dụng hàm .hex() để chuyển bản mã thành dạng chuỗi ký tự có thể đọc được, cắt đôi chuỗi và in ra Terminal để tiến hành phân tích sự khác biệt.

- Kết quả thực thi:

```
[AES-ECB]
Block 1: 02b0647de210da452e3bdcadc415814e
Block 2: 02b0647de210da452e3bdcadc415814e

[AES-CBC]
Block 1: 47747530a46ebd156c1da8763c54be1b
Block 2: 46807a7ec513e5e69d9256881a0f0c09

[AES-CFB]
Block 1: c6f55bf4ba891cc1491a83d7c9767f98
Block 2: c3f3561933409156300ebb2b74aca1c9

[AES-OFB]
Block 1: c6f55bf4ba891cc1491a83d7c9767f98
Block 2: fd0b617b99bebc3ac46d84cb18342642

[AES-CTR]
Block 1: 8dfccc178b264ecbe7d2008d7bdf6c25
Block 2: e27a5be88516c0a3682eb39cea266a85
```

*H*ình 2.2.2 Kết quả thực thi bao gồm cả thử nghiệm tất cả các Mode of Operation khác

Nhận xét:

- Chế độ ECB : Có thể thấy khối 1 và khối 2 của bản mã ECB giống hệt nhau từng ký tự. Nguyên nhân là do ECB mã hóa từng khối một cách hoàn toàn độc lập. Đúng như lưu ý đề cập trước đó, nếu dùng ECB trong thực tế, kẻ tấn công sẽ dễ dàng phát hiện ra quy luật của dữ liệu gốc ➔ làm lộ cấu trúc bản rõ.
- Chế độ CBC: 2 khối có bản rõ giống hệt nhau nhưng mã CBC của cả 2 lại khác nhau. Điều này là do CBC lấy khối bản rõ hiện tại đem XOR với khối bản mã ngay trước

nó rồi mới mã hóa – khối đầu tiên sẽ XOR với giá trị ngẫu nhiên ➔ che dấu cấu trúc lặp tốt

- Các chế độ khác: Đều khắc phục được nhược điểm của 2 chế độ trên với việc làm cho 2 khối đều khác nhau dựa trên nguyên lý biến mật mã khối (block cipher) thành mật mã luồng.

3. **Nhiệm vụ 2.3**

- Mã nguồn:

```
- from Crypto.Cipher import DES
-
- def get_bin(data):
-     # Chuyển dữ liệu bytes sang chuỗi nhị phân và thêm số 0 ở
  đầu cho đủ 64 bits
-     return bin(int.from_bytes(data,
  'big'))[2:].zfill(len(data)*8)
-
- def avalanche_test(key_str):
-     # Đảm bảo key có độ dài 8 bytes
-     key = key_str.encode('utf-8')
-     p1 = b'STAYHOME'
-     p2 = b'STAYHOMA' # khác nhau kí tự cuối
-
-     # Thực hiện mã hóa p1, p2 bằng DES-ECB
-     cipher = DES.new(key, DES.MODE_ECB)
-     c1 = cipher.encrypt(p1)
-     c2 = cipher.encrypt(p2)
-
-     # Chuyển kết quả sang dạng chuỗi nhị phân
-     b1 = get_bin(c1)
-     b2 = get_bin(c2)
-
-     # Đếm số bit khác nhau
-     diff_bits = sum(1 for bit1, bit2 in zip(b1, b2) if bit1 !=
  bit2)
-     total_bits = len(b1)
-     percent = (diff_bits / total_bits) * 100
-
-     print(f"--- Key: {key_str} ---")
-     print(f"Cipher 1 (bin): {b1}")
-     print(f"Cipher 2 (bin): {b2}")
```

```
-     print(f"Hamming Distance: {diff_bits}/{total_bits} bits")
-     print(f"Tỷ lệ thay đổi  : {percent:.2f}%\n")
-
-  # key mặc định
-  avalanche_test('87654321')
-
-  # key là Mã số sinh viên
-  avalanche_test('24521388')
```

Hình 2.3.1 Source code

- Thành phần chính:
  + Hàm get_bin: Xử lý dữ liệu thô. Do bản mã DES trả về là kiểu bytes, hàm này dùng int.from_bytes để ép kiểu và chuyển thành một chuỗi String nhị phân dài đúng 64 ký tự (chứa toàn '0' và '1').
  + Hàm avalanche_test: Hàm trung tâm, thực hiện toàn bộ vòng đời từ mã hóa đến tính toán khoảng cách Hamming.
- Các bước thực thi:
  + Khởi tạo Khóa và 2 bản rõ chỉ khác nhau 1 ký tự.
  + Đưa 2 bản rõ qua bộ mã hóa DES.MODE_ECB để thu về 2 bản mã C1 và C2
  + Gọi hàm get_bin để giải quyết C1 và C2 thành 2 chuỗi nhị phân B1, B2 dài 64-bits.
  + Sử dụng vòng lặp zip() trong Python để duyệt qua từng cặp bit ở cùng một vị trí của B1 và B2. Nếu bit nào khác nhau, bộ đếm (Hamming distance) sẽ cộng thêm 1.
  + Tính tỷ lệ % bit khác biệt và in ra kết quả.

- Kết quả chương trình:

```
PS C:\Users\admin\Documents\HK2_Nam2_2025-2026\NT101\TH\Lab01>  c:; cd 'c:\Users\ad
'; & 'd:\Python\python.exe' 'c:\Users\admin\.vscode\extensions\ms-python.debugpy-20
' '58348' '--' 'C:\Users\admin\Documents\HK2_Nam2_2025-2026\NT101\TH\Lab02_Nhom13_R
--- Key: 87654321 ---
Cipher 1 (bin): 1010000111000100110000010011000101011111100001100000010100110101
Cipher 2 (bin): 1110111000010011111010110000011000110000000011111100101101011111 0
Hamming Distance: 37/64 bits
Tỷ lệ thay đổi  : 57.81%

--- Key: 24521388 ---
Cipher 1 (bin): 0011100000011011111001101011101100100110110010010100101101000001
Cipher 2 (bin): 0001000010110110110110111010100110000111100110110011110000110011
Hamming Distance: 30/64 bits
Tỷ lệ thay đổi  : 46.88%

PS C:\Users\admin\Documents\HK2_Nam2_2025-2026\NT101\TH\Lab01>
```

Hình 2.3.2 Kết quả chương trình

Nhận xét:

- Với khóa mặc định: có 37 bit bị thay đổi trên 64 bit, chiếm tỷ lệ 57.81%
- Với khóa MSSV: có 30 bit bị thay đổi trên 64 bit chiếm tỷ lệ 46.88%

⇨ Dù sử dụng khóa nào, tỉ lệ lật bit giữa các bản mã luôn dao động và xấp xỉ ở mức 50%. Thuật toán đảm bảo rằng 1 sự xáo trộn nhỏ ở dữ liệu gốc cũng gây ra sự xáo trộn hoàn toàn ở bản mã, giúp hệ thống an toàn trước các phương pháp tấn công hiện đại.

4. Nhiệm vụ 2.6

- Mã nguồn

```python
import random

# TÍNH LŨY THỪA MODULE

def modular_pow(a, x, p):
    res = 1
    a = a % p
    while x > 0:
        if x % 2 == 1: # Nếu x lẻ, nhân a vào kết quả
            res = (res * a) % p
        x = x // 2     # Chia đôi x
        a = (a * a) % p # Bình phương a
    return res

# KIỂM TRA SỐ NGUYÊN TỐ (Miller-Rabin)
```

```
def miller_rabin(n, k=5):
    if n == 2 or n == 3: return True
    if n <= 1 or n % 2 == 0: return False

    # Viết n-1 dưới dạng 2^r * d
    r = 0
    d = n - 1
    while d % 2 == 0:
        r += 1
        d //= 2

    # Thử nghiệm k lần
    for _ in range(k):
        a = random.randrange(2, n - 1)
        x = modular_pow(a, d, n)
        if x == 1 or x == n - 1:
            continue
        for _ in range(r - 1):
            x = modular_pow(x, 2, n)
            if x == n - 1:
                break
        else:
            return False
    return True

def generate_n_bit_prime(bits):
    while True:
        # Sinh số lẻ ngẫu nhiên có độ dài 'bits'
        num = random.getrandbits(bits)
        num |= (1 << (bits - 1)) | 1
        if miller_rabin(num, 10):
            return num


# ƯỚC CHUNG LỚN NHẤT

def euclidean_gcd(a, b):
    while b != 0:
        a, b = b, a % b
```

```
    return a


# KẾT QUẢ THỰC THI (Main)

print("--- 1. TẠO SỐ NGUYÊN TỐ ---")
print(f"8-bit prime : {generate_n_bit_prime(8)}")
print(f"16-bit prime: {generate_n_bit_prime(16)}")
print(f"64-bit prime: {generate_n_bit_prime(64)}")

print("\n--- 2. TÌM 10 SỐ NGUYÊN TỐ NHỎ HƠN M_10 ---")
M10 = (2**89) - 1
print(f"Mersenne thứ 10 (M_10) = {M10}")
primes_below_m10 = []
num = M10 - 2 # Bắt đầu từ số lẻ ngay dưới M_10
while len(primes_below_m10) < 10:
    if miller_rabin(num, 10):
        primes_below_m10.append(num)
    num -= 2

for i, p in enumerate(primes_below_m10):
    print(f"Số thứ {i+1}: {p}")

print("\n--- 3. KIỂM TRA SỐ NGẪU NHIÊN < M_10 ---")
test_num = random.randrange(3, M10, 2)
is_p = miller_rabin(test_num, 10)
print(f"Số ngẫu nhiên: {test_num}")
print(f"Có phải số nguyên tố không? -> {is_p}")

print("\n--- 4. TÌM GCD CỦA 2 SỐ LỚN ---")
num1 = random.getrandbits(256)
num2 = random.getrandbits(256)
print(f"Số A: {num1}\nSố B: {num2}")
print(f"GCD(A, B) = {euclidean_gcd(num1, num2)}")

print("\n--- 5. LŨY THỪA MODULE ---")
print(f"7^40 mod 19 = {modular_pow(7, 40, 19)}")
```

Hình 2.6.1 Source code

- Các thành phần chính:

+ Hàm modular_pow: Tính lũy thừa module(bình phương rồi nhân liên tiếp)
+ Hàm miller_rabin: Kiểm tra số nguyên tố lớn bằng xác suất.
+ Hàm euclidean_gcd: Tìm ước chung lớn nhất bằng thuật toán Euclid.

- Các bước thực thi:

+ Tạo và kiểm tra số nguyên tố: Sinh ra một số lẻ ngẫu nhiên n-bit. Đưa số này vào hàm kiểm tra miller_rabin để thử nghiệm k=10 lần. Nếu qua bài test, số đó được giữ lại; nếu không, tiếp tục sinh số mới. Áp dụng tương tự bằng cách chạy lặp lùi (trừ 2) để tìm 10 số nguyên tố tiệm cận dưới của M10.
+ Tính GCD: sử dụng vòng lặp while b!= 0, liên tục thay thế a bằng b, và b bằng phần dư của a (mod b). Vòng lặp dừng lại khi số dư bằng 0, giá trị a cuối cùng chính là UCLN.
+ Tính lũy thừa Module với biểu thức a^x (mod p), hàm sẽ tách nhỏ số mũ x bằng cách chia đôi liên tục. Ở mỗi bước lặp sẽ áp dụng công thức sau với mục đích tránh việc lưu trữ con số quá lớn gây tràn ram:

$$(a \times b) \equiv [(a \bmod n) \times (b \bmod n)] \bmod n$$

- Kết quả chương trình:

```
PS C:\Users\admin\Documents\HK2_Nam2_2025-2026\NT101\TH\Lab01>  c:; cd 'c:\Users\admin\D
'; & 'd:\Python\python.exe' 'c:\Users\admin\.vscode\extensions\ms-python.debugpy-2025.18
' '58807' '--' 'C:\Users\admin\Documents\HK2_Nam2_2025-2026\NT101\TH\Lab02_Nhom13_Resour
--- 1. TẠO SỐ NGUYÊN TỐ ---
8-bit prime : 241
16-bit prime: 61717
64-bit prime: 12634406749139507947

--- 2. TÌM 10 SỐ NGUYÊN TỐ NHỎ HƠN M_10 ---
Mersenne thứ 10 (M_10) = 618970019642690137449562111
Số thứ 1: 618970019642690137449562091
Số thứ 2: 618970019642690137449562081
Số thứ 3: 618970019642690137449562063
Số thứ 4: 618970019642690137449562043
Số thứ 5: 618970019642690137449562013
Số thứ 6: 618970019642690137449562009
Số thứ 7: 618970019642690137449561847
Số thứ 8: 618970019642690137449561791
Số thứ 9: 618970019642690137449561671
Số thứ 10: 618970019642690137449561509

--- 3. KIỂM TRA SỐ NGẪU NHIÊN < M_10 ---
Số ngẫu nhiên: 384222540525838067218061893
Có phải số nguyên tố không? -> False

--- 4. TÌM GCD CỦA 2 SỐ LỚN ---
Số A: 816518566689836791044479160006286080143524915192695202701537544577517924695 26
Số B: 110805425874825362456389643789773723463188397596525316636276017569108118974907
GCD(A, B) = 1

--- 5. LŨY THỪA MODULE ---
7^40 mod 19 = 7
PS C:\Users\admin\Documents\HK2_Nam2_2025-2026\NT101\TH\Lab01> pacman -S mingw-w64-ucrt-
```

Hình 2.6.2 Kết quả chương trình

HẾT